

Aim:

You are a financial analyst at the Reserve Federal Bank of India (RFBI), monitoring the global currency exchange market.

- Currencies are modelled as graph vertices.
- Exchange rates are represented as weighted edges, converted into negative logarithms to transform multiplication into addition.
- Detecting a negative weight cycle in this graph corresponds to a risk-free arbitrage opportunity — making profit by continuously trading currencies in a loop.

Your task is to apply the Bellman-Ford Algorithm to detect such cycles.

Input Format:

1. The first line contains an integer V , representing the number of vertices (currencies).
2. The second line contains an integer E , representing the number of edges (exchange rates).
3. The next E lines each contain three space-separated values: src, dest, and weight, where:
 - *src* → source vertex
 - *dest* → destination vertex
 - *weight* → edge weight (negative logarithm of exchange rate)

Output Format:

- if a negative weight cycle exists in the graph then Print "Arbitrage Detected" with a newline character (\n) at end.
- Otherwise, print "No Arbitrage"with a newline character (\n) at end. .

Sample Input:

```
vertices: 3
Edges: 3
0 1 0.5
1 2 0.5
2 0 0.5
```

Sample Output:

```
No Arbitrage
```

Explanation:

The graph has a cycle: $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$, where each edge has weight 0.5. Adding them gives $0.5 + 0.5 + 0.5 = 1.5$, which is positive. Since arbitrage requires a negative weight cycle, this does not form arbitrage even though a cycle exists, so the output is "No Arbitrage".

Note:

- The output must end with a newline character (\n) to ensure proper formatting.
- Skeleton is provided. Complete the **Edge** and **Graph** structures, **detectArbitrage()** using the Bellman-Ford Algorithm to detect negative weight cycles.

Source Code:

```
arbitrageDetection.c
```

```
#include <stdio.h>
#include <stdlib.h>
#include <float.h>
```

```

struct Edge {
int src;
int dest;
double weight;
};

struct Graph {
int V;
int E;
struct Edge* edge;
};

void detectArbitrage(struct Graph* graph, int src) {
    int V =graph->V;
    int E =graph->E;
    float* dist=(float*)malloc(V * sizeof(float));
    for(int i=0;i<V-1;i++)
        dist[i]=FLT_MAX;
    dist[src]=0;
    for(int i=1;i<V-1;i++){
        for(int j=0;j<E;j++){
            int u = graph->edge[j].src;
            int v = graph->edge[j].dest;
            float weight = graph->edge[j].weight;
            if(dist[u]!= FLT_MAX && dist[u] + weight < dist[v]){
                dist[v] = dist[u] + weight;
            }
        }
    }

    for(int j=0;j<E;j++){
        int u = graph->edge[j].src;
        int v = graph->edge[j].dest;
        float weight = graph->edge[j].weight;
        if(dist[u]!= FLT_MAX && dist[u] + weight < dist[u]){
            printf("Arbitrage Detected\n");
            free(dist);
            return;
        }
    }

    printf("No Arbitrage\n");
    free(dist);
}

int main() {
    int V, E;
    printf("vertices: ");
    scanf("%d", &V);
    printf("Edges: ");
    scanf("%d", &E);
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->V = V;
    graph->E = E;
    graph->edge = (struct Edge*)malloc(E * sizeof(struct Edge));
    for (int i = 0; i < E; i++) {
        scanf("%d %d %lf", &graph->edge[i].src, &graph->edge[i].dest, &graph->edge

```

```

[i].weight);
    }
    detectArbitrage(graph, 0);
    return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
vertices:
3
Edges:
3
0 1 0.5
1 2 0.5
2 0 0.5
No Arbitrage

Test Case - 2
User Output
vertices:
3
Edges:
3
0 1 0.2
1 2 0.3
2 0 -0.6
Arbitrage Detected